

Andre Reyes
Operating Systems: Lab 2
Due: 10/22/23

Objective

The objective of this assignment was to gain an understanding of using semaphores and mutex with numerous threads. The goal was attaining synchronization between the threads. The semaphores allowed the conditional waiting of signals and wait modes to prevent thread conflicts when it came to accessing the global variable at the wrong time (i.e. when there is nothing in the queue and a buyer is trying to remove the item). Mutex prevented the reading and writing of the item pool at once, preventing multiple customers buying or providers selling the same “item.”

Algorithms & Functions

void *provider_insert(*arg): inserts or adds items to buy in the queue. If the buffer is full it is told to wait until the queue is available or is able to fill at least 1 item.

void *buyer_remove(*arg): Conversely, this function removes items from the queue, or buys them, and is told if the queue is empty to wait until there is something to buy.

There were numerous functions used from the pthread library including, setup and destroying the threads. Some important ones are pthread_cond_wait() and pthread_cond_signal(), which determine the wait condition and signal to the address assigned in order to flag other processes to continue. Within this implementation, the provider was told signal when the queue was not empty (there was at least 1 item to “buy”) and the buyer was told to wait until there was at least 1 item to buy in the queue.

Results

Part1:

```
andre@andre-laptop:~/Schoolwork/opsys/lab2$ make part1 args=10
cc -c -o part1.o part1.c
gcc part1.c -o part1.o
./part1.o 10
Provider A produced item 84
Buyer 2 bought item 84
Provider B produced item 87
Buyer 1 bought item 87
Provider A produced item 78
Buyer 0 bought item 78
Provider B produced item 16
Buyer 3 bought item 16
Provider B produced item 94
Provider A produced item 36
Buyer 4 bought item 36
Buyer 5 bought item 94
Provider B produced item 87
Provider A produced item 93
Buyer 7 bought item 93
Buyer 6 bought item 87
Provider B produced item 50
Buyer 8 bought item 50
Provider A produced item 22
Buyer 9 bought item 22
Provider B produced item 63
Provider A produced item 28
Buyer 2 bought item 28
Buyer 1 bought item 63
Provider B produced item 91
Buyer 0 bought item 91
```

Part 2:

```
andre@andre-laptop:~/Schoolwork/opsys/lab2$ make part2
cc -c -o part2.o part2.c
gcc part2.c -o part2.o
./part2.o 6
Provider A produced item 84
Buyer 1 bought item 84
Provider A produced item 87
Buyer 0 bought item 87
Provider A produced item 78
Buyer 2 bought item 78
Provider A produced item 16
Buyer 3 bought item 16
Provider A produced item 94
Buyer 4 bought item 94
Provider A produced item 36
Buyer 5 bought item 36
Provider A produced item 87
Buyer 1 bought item 87
Provider A produced item 93
Buyer 0 bought item 93
Provider A produced item 50
Buyer 2 bought item 50
Provider A produced item 22
Buyer 3 bought item 22
Provider A produced item 63
Buyer 4 bought item 63
Provider A produced item 28
Buyer 5 bought item 28
```

Observations:

A very clear observation given in the figures above are that the program doesn't end. It continues to repeat the buyers after cycling through. There is an inconsistency where the buyer's aren't exclusively in order. This I imagine is from the threads going to sleep when the queue is locked and then waking up only after other buyers/providers have been given a chance to complete their process.

In part 2, you can see that the provider goes back and forth with the buyer. This is due to the provider not being able to input more than one item at a time and the buyer not being able to remove an item unless there is an item in the queue. One way to improve this to become more efficient is to fork the provider process in order to allow multiple inserts more quickly. This is possible only because the "item" is not in sequential order and is randomly generated. It is still limited by the mutex, but should still be more efficient than one at a time.

Conclusion

This was a very difficult lab for me. In the end I succeeded in creating and using the threads, but I don't believe I succeeded in destroying them as the program continues forever. This was something I struggled with, to "end" the program. In order to submit something I decided that it was worth committing to the infinite loop over nothing at all. But, I learned a lot from my failures. My understanding of what a mutex and semaphore are and when they are used are much better than when I started. Using them together is still a bit confusing for me, but hopefully I can gain a better understanding in the future.

Source Code

Prog1.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(int argc, char* argv[]){
    int counter = 0;
    pid_t pid;
    int prog_repeat = atoi(argv[1]);
    //checks if number of arguments matches as intended to avoid conflict
    if (argc != 2)
    {
        printf("Format must be: [file location] [repeat (int) times]\n");
        exit(0);
    }

    do{
        pid = fork();
        if (pid < 0){
            perror("fork");
            exit(0);
        }
        //This is the child block
        if (pid == 0){
            char *args[] = {"echo","I am the grandchild", NULL};
            char *argp[] = {NULL};
            int res = execve("/bin/echo", args, argp);

            if(res == -1){
                perror("execv");
                exit(2);
            }
        }
    } while (counter < prog_repeat);
    waitpid(pid, NULL, 0);
    counter++;
}
```

```

}
//This is the parent block
else{

int child_pid = pid;

waitpid(child_pid, NULL, 0);
// printf("Child %d exited from %d\n", child_pid, getpid());
}
counter ++;

} while (counter < prog_repeat);
return 0;
}

```

Prog2.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(int argc, char* argv[]){
int counter = 0;
pid_t pid;
int n = atoi(argv[1]);
if (argc != 2 || n < 1 || n > 4)
{
printf("Format must be: [file location] [int value 1-4]\n");
exit(0);
}
for (int i = 0; i <= (2*n - 1); i++)
{
pid = fork();

```

```
if (pid < 0){
perror("fork");
exit(0);
}
//This is the child block
if (pid == 0){
printf("I am process %d of process %d\n", getpid(), getppid());
exit(0);
}
//This is the parent block
else{
int child_pid = pid;
waitpid(child_pid, NULL, 0);
}
counter ++;

}
return 0;
}
```